REAL TIME GROUNDWATER RESOURCE EVALUATION USING DWLR DATA

Daily Work Report & Frontend Development Theory

Coverage Period: June 16 – June 20

Module: Frontend Development (React.js)

June 16 — Dashboard Development

Activity: Dashboard Development

On June 16, the main Dashboard for the Real Time Groundwater Resource Evaluation System is developed. The dashboard acts as the central control panel where authenticated users view live DWLR (Digital Water Level Recorder) readings, station-wise groundwater levels, alerts, and summary statistics immediately after login.

What is Dashboard Development?

Dashboard Development involves designing and building a data-driven landing page that consolidates real-time and historical groundwater data into an easily readable visual interface. Since DWLR stations continuously transmit water level, rainfall, and battery status data, the dashboard must present this information using cards, charts, tables, and status indicators that update dynamically, giving hydrogeologists and field engineers a quick overview of groundwater conditions across all monitored stations.

Key Tasks Involved:

- Design the dashboard layout using a responsive grid of React components (cards, panels, charts).
- Display summary widgets — total DWLR stations, active/inactive stations, average groundwater level, and critical alerts.
- Integrate a charting library (e.g., Recharts / Chart.js) to plot water level trends over time for each station.
- Fetch real-time/near real-time data from the Spring Boot backend using Axios/Fetch and refresh it at set intervals.
- Implement station-wise filtering so users can view data for a specific district, block, or DWLR station ID.
- Add color-coded status indicators (Normal / Warning / Critical) based on groundwater depletion thresholds.
- Ensure role-based dashboard views — Admin sees all stations, Field Engineer sees assigned stations, Viewer sees read-only summary.
- Apply responsive design so the dashboard remains usable on tablets and mobile devices used in the field.

Expected Deliverable:

Dashboard UI — A fully functional, data-driven dashboard that visually presents real-time DWLR groundwater data through charts, summary cards, and station-wise filters, accessible according to the logged-in user's role.

Technology	Purpose
React.js	Frontend UI and component structure for the dashboard
Recharts / Chart.js	Visualizing groundwater level trends and station statistics
Axios / Fetch API	Fetching live DWLR data from the Spring Boot backend
React Router	Navigation between dashboard, profile, and report sections

CSS / Tailwind	Responsive grid layout and status-based color coding
Spring Boot (Backend)	Supplies aggregated DWLR sensor data to the dashboard APIs

---

June 17 — CRUD Form Development

Activity: CRUD Form Development

On June 17, the CRUD (Create, Read, Update, Delete) forms for the Real Time Groundwater Resource Evaluation System are developed. These forms allow authorized users to add new DWLR station records, update existing readings or station metadata, and remove obsolete entries from the system.

What is CRUD Form Development?

CRUD Form Development involves building reusable input forms that handle the four basic database operations — Create, Read, Update, and Delete — for core entities of the system such as DWLR Station, Groundwater Reading, and User. Because groundwater records must be accurate and traceable, every form requires strict validation, confirmation steps for destructive actions, and clear feedback to the user.

Key Tasks Involved:

- Design the 'Add DWLR Station' form with fields such as Station ID, Location, Latitude/Longitude, District, Block, and Installation Date.
- Design the 'Add/Edit Groundwater Reading' form with fields such as Station ID, Water Level (m), Date/Time of Recording, and Sensor Battery Status.
- Implement client-side validation for numeric ranges, mandatory fields, date formats, and duplicate station IDs.
- Connect Create and Update forms to the respective Spring Boot REST endpoints (POST and PUT) using Axios.
- Implement Delete functionality with a confirmation dialog to prevent accidental removal of station or reading records.
- Pre-fill Update forms with existing record data fetched from the backend before allowing edits.
- Display success and error toasts/alerts after each Create, Update, or Delete operation.
- Maintain consistent form styling and layout across all CRUD modules for a uniform user experience.

Expected Deliverable:

Forms Completed — A complete set of validated CRUD forms enabling authorized users to add, update, and delete DWLR station and groundwater reading records, fully integrated with the backend REST APIs.

Technology	Purpose
React.js	Building reusable, component-based CRUD form UI
React Hook Form / useState	Managing form state and field-level validation
Axios / Fetch API	Sending Create, Update, and Delete requests to the backend

CSS / Tailwind	Consistent styling across all CRUD forms
React Toastify (or similar)	Showing success/error notifications after form actions
Spring Boot (Backend)	Processing CRUD requests and persisting changes to the database

---

June 18 — Table & Search Features

Activity: Table & Search Features

On June 18, the data listing tables and search/filter features are developed for the Real Time Groundwater Resource Evaluation System. This functionality allows users to browse, search, sort, and filter through large volumes of DWLR station and groundwater reading records efficiently.

What are Table & Search Features?

Table and Search Features involve building dynamic, paginated data tables that display groundwater records fetched from the backend, along with search bars and filter controls that let users quickly locate specific stations, date ranges, or readings that meet certain thresholds. Given that DWLR stations generate continuous time-series data, efficient listing, sorting, and filtering are essential for usability.

Key Tasks Involved:

- Build a reusable data table component to display DWLR station and groundwater reading records.
- Implement pagination to handle large datasets without overloading the browser or the API.
- Add a search bar to filter records by Station ID, location, or district in real time.
- Implement column-based sorting (e.g., sort by water level, date, or station name) ascending/descending.
- Add advanced filters — date range picker, district/block dropdown, and water level threshold filters.
- Highlight rows with critical groundwater levels using conditional row styling.
- Connect search and filter parameters to the Spring Boot backend via query parameters for server-side filtering where needed.
- Add an export option (CSV/Excel) for filtered groundwater data for offline reporting.

Expected Deliverable:

Data Listing — A fully functional, paginated, searchable, and sortable data table that allows users to efficiently browse and filter DWLR station and groundwater reading records.

Technology	Purpose
React.js	Building the dynamic table and search/filter UI components
Axios / Fetch API	Fetching filtered/paginated data from the backend
React Table / TanStack Table	Handling sorting, pagination, and column management
CSS / Tailwind	Styling tables, search bars, and conditional row highlighting
Spring Boot (Backend)	Providing paginated and filtered groundwater data via REST APIs

---

June 19 — Frontend Testing

Activity: Frontend Testing

On June 19, the frontend modules built so far — Login, Registration, Dashboard, CRUD Forms, and Table/Search features — are tested for the Real Time Groundwater Resource Evaluation System. This ensures that the user interface behaves correctly, displays accurate data, and handles errors gracefully before backend integration is finalized.

What is Frontend Testing?

Frontend Testing is the process of verifying that all React components, forms, and pages function as intended across different scenarios, devices, and user roles. For a groundwater monitoring system, this includes confirming that data is rendered correctly, validations work as expected, and the interface remains responsive and reliable for field engineers using mobile devices near DWLR stations.

Key Tasks Involved:

- Perform unit testing of individual React components (Login form, Registration form, Dashboard cards) using Jest and React Testing Library.
- Conduct integration testing to verify that forms correctly send data to and receive responses from the Spring Boot APIs.
- Test role-based UI rendering — confirm Admin, Field Engineer, and Viewer roles see the correct dashboard views and permissions.
- Validate all form error states — invalid login, duplicate registration, invalid CRUD inputs, and empty search results.
- Test table pagination, sorting, and filtering logic with sample DWLR datasets.
- Perform cross-browser and cross-device testing (desktop, tablet, mobile) for responsive layout consistency.
- Check loading states, API failure handling, and network-timeout behavior on slow field connections.
- Document identified bugs and UI inconsistencies for resolution in the bug-fixing phase.

Expected Deliverable:

Frontend Review — A documented testing report confirming that all frontend modules function correctly, handle errors gracefully, and render appropriately across devices and user roles, along with a list of issues to be resolved.

Technology	Purpose
Jest	Unit testing of React components and utility functions
React Testing Library	Testing component rendering and user interactions
Postman (cross-check)	Verifying API responses consumed by the frontend
Chrome DevTools	Responsive design testing and performance/network inspection
React.js	The frontend codebase under test

June 20 — Spring Boot Project Setup

Activity: Spring Boot Project Setup

On June 20, the backend project for the Real Time Groundwater Resource Evaluation System is initialized using Spring Boot. This marks the beginning of backend development, laying the foundation on which authentication, CRUD, and DWLR data APIs will be built in the upcoming days.

What is Spring Boot Project Setup?

Spring Boot Project Setup involves initializing a new backend project with the required dependencies, folder structure, and configuration needed to support a layered architecture (Controller, Service, Repository, Entity). For a groundwater monitoring system, the backend must be structured to handle continuous DWLR sensor data ingestion, user authentication, and secure REST API exposure to the React frontend.

Key Tasks Involved:

- Generate the Spring Boot project using Spring Initializr with required dependencies: Spring Web, Spring Data JPA, Spring Security, MySQL/PostgreSQL Driver, and Lombok.
- Define the standard layered package structure — controller, service, repository, entity, dto, and config packages.
- Configure the application.properties (or application.yml) file with database connection details, server port, and JWT secret placeholders.
- Set up the database connection for the groundwater monitoring schema (DWLR stations, readings, users).
- Enable CORS configuration to allow the React frontend (running on a different port) to communicate with the backend.
- Set up base exception handling and a standard API response structure for consistency across all endpoints.
- Verify the project runs successfully and the database connects without errors using a basic health-check endpoint.
- Initialize version control (Git) for the backend project and push the base setup to the repository.

Expected Deliverable:

Backend Setup — A fully initialized and runnable Spring Boot project with proper layered structure, database connectivity, and CORS configuration, ready for entity creation and API development in the following days.

Technology	Purpose
Spring Boot	Core backend framework for the groundwater evaluation system
Spring Data JPA	ORM layer for database interaction with DWLR data tables
MySQL / PostgreSQL	Relational database storing station and reading records
Spring Security (initial config)	Foundation for upcoming authentication and RBAC
Lombok	Reducing boilerplate code in entity and DTO classes

Maven / Gradle	Dependency management and project build tool